

YOWYOB ERP

Comptabilité Générale et Analytique

Guide d'implémentation Mode hors ligne (Offline)

Spécification technique Backend et Frontend

Version 1.0
Date Juillet 2026
Public Développeurs frontend, développeurs backend, architectes
Statut Document de référence pour la mise en œuvre

Ce document décrit l'architecture, les contrats API et les procédures d'implémentation pour un fonctionnement *online-first avec repli offline*.

Contents

1	Introduction	3
1.1	Objectif	3
1.2	Périmètre	3
1.3	Principes directeurs	3
2	Architecture globale	5
2.1	Vue d'ensemble	5
2.2	Composants frontend	5
2.3	Composants backend	6
3	Détection de l'état réseau	7
3.1	Signaux utilisés	7
3.2	Distinction erreur réseau vs erreur métier	7
3.3	Algorithme de décision	7
4	Implémentation Frontend	8
4.1	Stockage local : IndexedDB	8
4.1.1	Schéma Dexie (version 1)	8
4.1.2	Table <code>entities</code>	8
4.1.3	Table <code>outbox</code>	8
4.2	Flux de lecture	9
4.3	Flux d'écriture	9
4.4	Règle d'or	9
4.5	Moteur de synchronisation	9
4.5.1	Algorithme flush	9
4.5.2	Backoff	10
4.6	Intégration dans l'application existante	10
4.6.1	Layout dashboard	10
4.6.2	<code>useAutoRefresh</code>	10
4.6.3	Indicateur utilisateur	10
4.7	Pilote : écritures analytiques	10
5	Implémentation Backend	11
5.1	Endpoints requis — Comptabilité analytique	11
5.1.1	Écritures analytiques	11
5.1.2	Journaux analytiques	11
5.2	Contrat de création avec idempotence	11
5.2.1	Requête POST	11
5.2.2	Réponse succès	12

5.2.3	Réponse idempotente (rejeu)	12
5.3	Versionnement et conflits	12
5.4	Politique de conflit recommandée (ERP comptable)	13
5.5	Health check	13
6	Garanties et limites	14
6.1	Ce qui est garanti (niveau production)	14
6.2	Ce qui n'est pas garanti à 100%	14
6.3	Niveaux de fiabilité	14
7	Plan de déploiement	15
7.1	Phase 1 — Fondations (actuelle)	15
7.2	Phase 2 — Lecture offline CG	15
7.3	Phase 3 — Écriture offline CG	15
7.4	Phase 4 — PWA	15
8	Tests et validation	16
8.1	Scénarios de test	16
8.2	Critères d'acceptation	16
9	Annexes	17
9.1	Arborescence fichiers frontend	17
9.2	Événements internes	17
9.3	Références	17

Chapter 1

Introduction

1.1 Objectif

Ce guide définit comment permettre à un utilisateur de continuer à travailler dans l'application web lorsque la connexion réseau est indisponible, puis de retrouver automatiquement l'ensemble de son travail dans l'application une fois la connexion rétablie.

Le principe retenu est le suivant :

- **En ligne** : communication systématique avec le backend (source de vérité serveur).
- **Hors ligne** : bascule transparente vers une mémoire locale persistante.
- **Retour en ligne** : synchronisation automatique de la file d'attente vers le backend.

1.2 Périmètre

Inclus	Détection réseau, cache local, file d'attente (out-box), moteur de sync, contrat API backend, gestion des conflits, indicateur discret, pilote écritures analytiques
Hors périmètre initial	PWA complète (Service Worker), sync multi-appareils temps réel, offline sur pièces jointes volumineuses

1.3 Principes directeurs

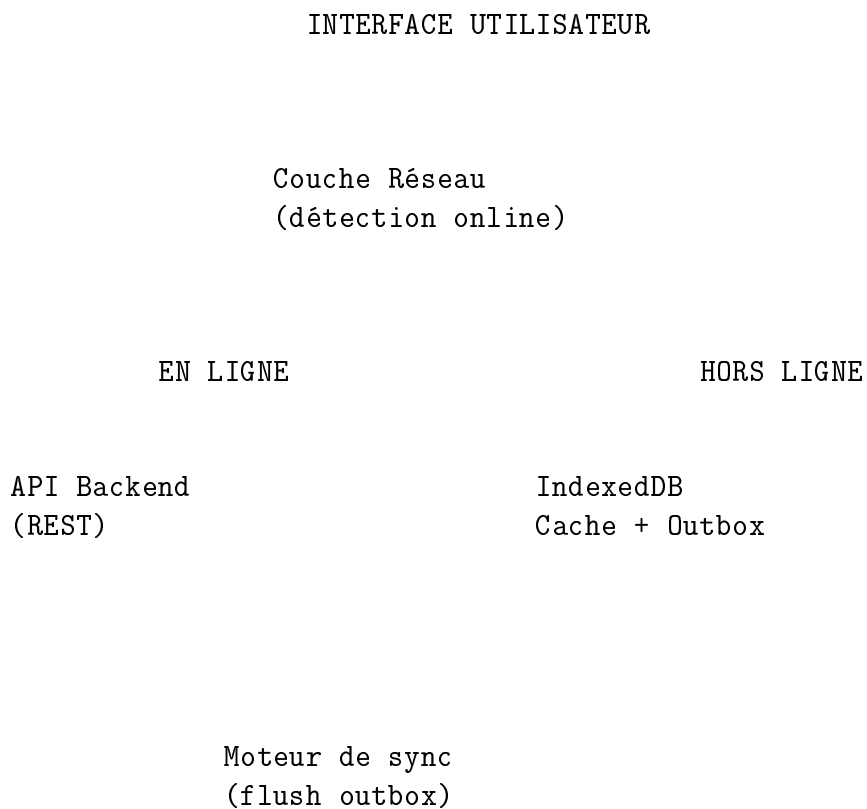
1. **Online-first** : tant que le réseau et l'API répondent, toute lecture et écriture passe par le backend.
2. **Offline en dernier recours** : le stockage local n'est activé que si la connexion ou l'API est indisponible.
3. **Transparence** : l'utilisateur n'est pas bloqué ; un indicateur discret signale les éléments en attente de synchronisation.
4. **Pas de perte** : aucune donnée locale n'est supprimée avant accusé de réception serveur.

5. **Idempotence** : le backend doit accepter le rejeu d'opérations sans créer de doublons.

Chapter 2

Architecture globale

2.1 Vue d'ensemble



2.2 Composants frontend

Composant	Rôle
network-status	Détecte online/offline via <code>navigator.onLine</code> , événements navigateur et échecs API consécutifs
offline/db	Schéma IndexedDB (Dexie) : entités, outbox, méta-données

<code>offline/outbox</code>	Enregistre chaque mutation (CREATE, UPDATE, DELETE) en attente
<code>offline/sync-engine</code>	Rejoue la file d'attente au retour en ligne
<code>offline/handlers</code>	Adaptateurs par type d'entité (écritures analytiques, etc.)
<code>use-network-status</code>	Hook React exposant <code>isOnline</code> , <code>pendingCount</code>
<code>OfflineProvider</code>	Provider global monté dans le layout dashboard

2.3 Composants backend

Composant	Rôle
API REST CRUD	Endpoints standards par entité métier
Idempotence	Acceptation de <code>clientMutationId</code> ou en-tête <code>Idempotency-Key</code>
Versionnement	Champ <code>updatedAt</code> et/ou <code>version</code> sur chaque entité
Sync delta	Endpoint <code>GET ?since=timestamp</code> (phase 2)
Gestion conflits	Réponse HTTP 409 avec détail structuré
Audit	Journal des opérations comptables

Chapter 3

Détection de l'état réseau

3.1 Signaux utilisés

1. `navigator.onLine` : détection immédiate de coupure réseau locale.
2. Événements `window.online` et `window.offline`.
3. Compteur d'échecs API consécutifs (`timeout`, `Failed to fetch`, status 0).
4. Repassage en mode online uniquement après un appel API réussi.

3.2 Distinction erreur réseau vs erreur métier

Type	Exemples	Comportement
Réseau	Timeout, fetch failed, status 0	Bascule offline, cache + outbox
Métier	422 validation, 403 interdit	Message utilisateur, pas de bascule offline
Serveur	500, 502, 503	Retry avec backoff, pas de bascule immédiate
Conflit	409	Marquer conflit, conserver donnée locale

3.3 Algorithme de décision

```
function shouldUseOffline(): boolean {  
  if (!navigator.onLine) return true;  
  if (consecutiveNetworkFailures >= 2) return true;  
  return false;  
}
```


Chapter 4

Implémentation Frontend

4.1 Stockage local : IndexedDB

Ne pas utiliser `localStorage` comme stockage principal offline (limite 5 Mo, pas de requêtes structurées).

4.1.1 Schéma Dexie (version 1)

```
entities  : key, entity, entityId, updatedAt
outbox    : id, entity, entityId, status, createdAt
meta      : key
```

4.1.2 Table entities

Cache des données métier pour consultation hors ligne.

```
{
  key: "écriture_analytique:ea-uuid",
  entity: "écriture_analytique",
  entityId: "ea-uuid",
  data: { ... },
  updatedAt: "2026-07-08T12:00:00Z",
  syncStatus: "synced" | "pending" | "local_only"
}
```

4.1.3 Table outbox

File d'attente des opérations à envoyer au backend.

```
{
  id: "op-uuid",
  entity: "écriture_analytique",
  action: "CREATE" | "UPDATE" | "DELETE",
  entityId: "ea-uuid",
  payload: { ... },
  clientMutationId: "cm-uuid",
  createdAt: "2026-07-08T12:00:00Z",
}
```

```

    status: "pending" | "syncing" | "done" | "failed" | "conflict",
    retries: 0,
    lastError: null
  }

```

4.2 Flux de lecture

1. Si **en ligne** : appel API → mise à jour cache IndexedDB → affichage.
2. Si **hors ligne** : lecture cache IndexedDB → affichage.
3. Si API échoue (réseau) : lecture cache en repli, bascule offline.

4.3 Flux d'écriture

1. Validation métier légère côté frontend (champs obligatoires, formats).
2. Si **en ligne** : envoi API → si succès, mise à jour cache ; si échec réseau → étape 3.
3. Si **hors ligne** ou échec réseau :
 - Sauvegarde immédiate dans IndexedDB.
 - Ajout dans outbox (status `pending`).
 - Mise à jour UI (optimistic).
4. Au retour en ligne : `syncEngine.flush()` rejoue l'outbox dans l'ordre chronologique.

4.4 Règle d'or

Ne jamais supprimer une entrée de l'outbox ni une entité locale tant que le serveur n'a pas confirmé la réception (HTTP 200/201).

4.5 Moteur de synchronisation

4.5.1 Algorithme flush

1. Récupérer les opérations outbox où `status = pending` ou `failed` (`retries < 5`), triées par `createdAt`.
2. Pour chaque opération :
 - Passer en `syncing`.
 - Appeler le handler de l'entité (`push`).
 - Si succès : `done`, mettre à jour `entities.syncStatus = synced`.
 - Si erreur réseau : `pending`, incrémenter `retries`, arrêter le flush.
 - Si 409 conflit : `conflict`, notifier discrètement.
 - Si 422 métier : `failed`, conserver localement.
3. Émettre événement `sync:complete` avec le nombre d'opérations synchronisées.

4.5.2 Backoff

Tentative	Délai avant retry
1	0 s (immédiat au online)
2	2 s
3	5 s
4	15 s
5	30 s

4.6 Intégration dans l'application existante

4.6.1 Layout dashboard

Monter `OfflineProvider` qui :

- initialise `IndexedDB` ;
- écoute `online/offline` ;
- lance le `flush` au retour en ligne ;
- expose le nombre d'opérations en attente.

4.6.2 `useAutoRefresh`

Désactiver le polling quand hors ligne :

```
useAutoRefresh(callback, deps, { enabled: isOnline });
```

4.6.3 Indicateur utilisateur

Point discret dans le header :

- Vert : en ligne, rien en attente.
- Orange : hors ligne OU opérations en attente de sync.
- Tooltip : « 3 élément(s) en attente de synchronisation ».

4.7 Pilote : écritures analytiques

Première entité migrée :

- Remplacement de `localStorage` par `IndexedDB`.
- Migration automatique des données existantes au premier chargement.
- Chaque `CREATE/UPDATE/DELETE` alimente l'outbox.
- Handler sync prêt à brancher l'API backend quand disponible.

Chapter 5

Implémentation Backend

5.1 Endpoints requis — Comptabilité analytique

5.1.1 Écritures analytiques

Méthode	Endpoint	Description
GET	/api/accounting/analytic-entries	Liste (filtres : statut, période)
GET	/api/accounting/analytic-entries/{id}	Détails
POST	/api/accounting/analytic-entries	Création (accepte clientId)
PUT	/api/accounting/analytic-entries/{id}	Mise à jour
DELETE	/api/accounting/analytic-entries/{id}	Suppression
POST	/api/accounting/analytic-entries/{id}/validate	Validation
POST	/api/accounting/analytic-entries/{id}/reject	Rejet
GET	/api/accounting/analytic-entries/{id}/series	Séries de la période

5.1.2 Journaux analytiques

Méthode	Endpoint	Description
GET	/api/accounting/analytic-journals	Liste
POST	/api/accounting/analytic-journals	Création
PUT	/api/accounting/analytic-journals/{id}	Mise à jour
DELETE	/api/accounting/analytic-journals/{id}	Suppression

5.2 Contrat de création avec idempotence

5.2.1 Requête POST

POST /api/accounting/analytic-entries

Headers:

Authorization: Bearer {token}
X-Organization-Id: {orgId}
X-Tenant-Id: {tenantId}

Idempotency-Key: {clientMutationId}

Body:

```
{
  "clientId": "ea-uuid-genere-cote-client",
  "journalId": "...",
  "dateEffet": "2026-07-08",
  "numeroPiece": "ECRIT-ANALYTIQUE-2026-0001",
  "libelleOperation": "...",
  "centreDestinationId": "...",
  "axeId": "...",
  "exerciceAnalytiqueId": "...",
  "natureChargeId": "...",
  "montant": 450000,
  "lignes": [ ... ],
  "origine": "MANUELLE"
}
```

5.2.2 Réponse succès

HTTP 201 Created

```
{
  "success": true,
  "data": {
    "id": "serveur-uuid",
    "clientId": "ea-uuid-genere-cote-client",
    "version": 1,
    "updatedAt": "2026-07-08T12:00:00Z",
    ...
  }
}
```

5.2.3 Réponse idempotente (rejeu)

Si Idempotency-Key déjà traité :

HTTP 200 OK

```
{
  "success": true,
  "data": { ... entité existante ... },
  "message": "ALREADY_PROCESSED"
}
```

5.3 Versionnement et conflits

Chaque entité modifiable doit exposer :

- **version** (entier incrémenté) ou **updatedAt** (ISO 8601).

- Le PUT/PATCH doit accepter If-Match: `{version}` ou `expectedVersion` dans le body.

En cas de conflit :

HTTP 409 Conflict

```
{
  "success": false,
  "code": "VERSION_CONFLICT",
  "message": "L'entité a été modifiée par un autre utilisateur",
  "data": {
    "serverVersion": { ... état serveur ... },
    "clientVersion": { ... état client ... }
  }
}
```

5.4 Politique de conflit recommandée (ERP comptable)

Situation	Politique
Brouillon local jamais synchronisé	Client gagne (première sync)
Écriture déjà validée côté serveur	Serveur gagne
Modification concurrente sur brouillon	Résolution manuelle (UI)
Suppression serveur + modification locale	Conflit, conservation locale

5.5 Health check

Endpoint léger pour vérifier la disponibilité API au retour en ligne :

GET /api/health

Response: { "status": "UP" }

Chapter 6

Garanties et limites

6.1 Ce qui est garanti (niveau production)

- Survit au rechargement de page (IndexedDB persistant).
- Survit à une coupure réseau pendant la saisie.
- Sync automatique au retour en ligne.
- Pas de doublon si idempotence backend respectée.
- Opérations en attente visibles (compteur discret).

6.2 Ce qui n'est pas garanti à 100%

- Utilisateur vide le cache navigateur → perte du local non synchronisé.
- Navigation privée sans persistance.
- Multi-appareils sans sync temps réel.
- Quota IndexedDB dépassé (gros fichiers).
- Conflits non résolus manuellement.

6.3 Niveaux de fiabilité

Niveau	Description
A	localStorage seul, pas de backend → risque élevé
B	IndexedDB + outbox, un navigateur → très bon
C	B + backend idempotent + conflits gérés → quasi zéro perte
D	Absolu sur le web → impossible

Chapter 7

Plan de déploiement

7.1 Phase 1 — Fondations (actuelle)

- IndexedDB + outbox + détection réseau.
- OfflineProvider dans le layout.
- Pilote écritures analytiques.
- Migration localStorage → IndexedDB.
- Indicateur discret dans le header.

7.2 Phase 2 — Lecture offline CG

- Cache des listes comptabilité générale (écritures, plan comptable, journaux).
- Lecture cache en repli hors ligne.

7.3 Phase 3 — Écriture offline CG

- APIs backend avec idempotence.
- Handlers sync pour écritures comptables (brouillons d'abord).

7.4 Phase 4 — PWA

- Service Worker pour shell applicatif.
- Ouverture de l'app sans réseau après première visite.

Chapter 8

Tests et validation

8.1 Scénarios de test

1. **Saisie hors ligne** : couper réseau, créer écriture, recharger page → donnée présente.
2. **Retour en ligne** : rétablir réseau → outbox vidée, compteur à 0.
3. **Échec API** : API down mais `onLine=true` → bascule offline après 2 échecs.
4. **Idempotence** : rejouer même `clientMutationId` → pas de doublon.
5. **Conflit 409** : modification concurrente → statut conflit, donnée locale conservée.
6. **Migration** : données `localStorage` existantes → présentes dans `IndexedDB`.

8.2 Critères d'acceptation

- Aucun loader bloquant en mode offline.
- Polling AJAX désactivé hors ligne.
- Indicateur visible si opérations en attente.
- Aucune perte sur rechargement page en offline.
- Sync silencieuse au retour en ligne (toast optionnel).

Chapter 9

Annexes

9.1 Arborescence fichiers frontend

```
lib/offline/  
  db.ts  
  network-status.ts  
  outbox.ts  
  sync-engine.ts  
  migrate-local-storage.ts  
  handlers/  
    ecriture-analytique-sync.ts  
hooks/  
  use-network-status.ts  
  use-offline-sync.ts  
components/offline/  
  offline-provider.tsx  
  offline-status-indicator.tsx
```

9.2 Événements internes

Événement	Déclencheur
network:offline	Coupure réseau ou 2+ échecs API
network:online	Événement online + API OK
sync:start	Début flush outbox
sync:complete	Fin flush (avec count)
sync:conflict	Opération en conflit 409

9.3 Références

- MDN — `navigator.onLine`, IndexedDB API
- Dexie.js — wrapper IndexedDB
- Pattern Outbox — microservices / offline sync

- RFC — Idempotency-Key (pratique courante REST)